

1. Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples.

```
import pandas as pd

# -----
# Dataset: Email Spam Detection
# -----
data = [
    ['Yes', 'Yes', 'Yes', 'No', 'Yes', 'Yes'],
    ['Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes'],
    ['No', 'No', 'No', 'No', 'No', 'No'],
    ['Yes', 'No', 'Yes', 'No', 'Yes', 'Yes'],
    ['No', 'Yes', 'No', 'Yes', 'No', 'No'],
    ['Yes', 'Yes', 'Yes', 'No', 'No', 'Yes']
]

columns = [
    'ContainsLink',
    'ContainsOffer',
    'UnknownSender',
    'Attachment',
    'ManyRecipients',
    'Spam'
]

df = pd.DataFrame(data, columns=columns)

print("Training Dataset:\n")
print(df)

# -----
# FIND-S Algorithm
# -----

def find_s(data):
    # Initialize hypothesis with None
    hypothesis = [None] * (len(data[0]) - 1)

    print("\nInitial Hypothesis:")
    print(hypothesis)

    for i, row in enumerate(data):
        if row[-1] == 'Yes': # Positive example
            if hypothesis[0] is None:
                hypothesis = row[:-1].copy()
```

```

else:
for j in range(len(hypothesis)):
if hypothesis[j] != row[j]:
hypothesis[j] = '?'

print(f"\nAfter Positive Example {i+1}:")
print(hypothesis)

return hypothesis

# Run FIND-S
final_hypothesis = find_s(data)

print("\nFinal Most Specific Hypothesis:")
print(final_hypothesis)

```

output:

```

... Training Dataset:

```

	ContainsLink	ContainsOffer	UnknownSender	Attachment	ManyRecipients	Spam
0	Yes	Yes	Yes	No	Yes	Yes
1	Yes	Yes	Yes	Yes	Yes	Yes
2	No	No	No	No	No	No
3	Yes	No	Yes	No	Yes	Yes
4	No	Yes	No	Yes	No	No
5	Yes	Yes	Yes	No	No	Yes

```

Initial Hypothesis:
[None, None, None, None, None]

After Positive Example 1:
['Yes', 'Yes', 'Yes', 'No', 'Yes']

After Positive Example 2:
['Yes', 'Yes', 'Yes', '?', 'Yes']

After Positive Example 4:
['Yes', '?', 'Yes', '?', 'Yes']

After Positive Example 6:
['Yes', '?', 'Yes', '?', '?']

Final Most Specific Hypothesis:
['Yes', '?', 'Yes', '?', '?']

```

2. For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in python to output a description of the set of all hypotheses consistent with the training examples

```

data = [
['Sunny','Warm','Normal','Strong','Warm','Same','Yes'],
['Sunny','Warm','High','Strong','Warm','Same','Yes'],
['Rainy','Cold','High','Strong','Warm','Change','No'],

```

```
['Sunny','Warm','High','Strong','Cool','Change','Yes']
]
```

```
concepts = [i[:-1] for i in data]
target = [i[-1] for i in data]
```

```
S = concepts[0]
G = [['?' for _ in S] for _ in S]
```

```
for i, h in enumerate(concepts):
    if target[i] == 'Yes':
        for j in range(len(S)):
            if h[j] != S[j]:
                S[j] = '?'
                G[j][j] = '?'
            else:
                for j in range(len(S)):
                    if h[j] != S[j]:
                        G[j][j] = S[j]
                    else:
                        G[j][j] = '?'
```

```
G = [g for g in G if g != ['?'] * len(S)]
```

```
print("Specific Hypothesis:")
print(S)
```

```
print("\nGeneral Hypothesis:")
for g in G:
    print(g)
```

output:

```
... Specific Hypothesis:
    ['Sunny', 'Warm', '?', 'Strong', '?', '?']

    General Hypothesis:
    ['Sunny', '?', '?', '?', '?', '?']
    ['?', 'Warm', '?', '?', '?', '?']
```

3. Demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder
```

```

# Dataset
data = {
'Outlook':['Sunny','Sunny','Overcast','Rain','Rain','Rain','Overcast','Sunny'],
'Temp':['Hot','Hot','Hot','Mild','Cool','Cool','Cool','Mild'],
'Humidity':['High','High','High','High','Normal','Normal','Normal','High'],
'Wind':['Weak','Strong','Weak','Weak','Weak','Strong','Strong','Weak'],
'Play':['No','No','Yes','Yes','Yes','No','Yes','No']
}

# Encode data
le = LabelEncoder()
X = []
for key in ['Outlook','Temp','Humidity','Wind']:
X.append(le.fit_transform(data[key]))
X = list(zip(*X))

y = LabelEncoder().fit_transform(data['Play'])

# Train ID3 Decision Tree
model = DecisionTreeClassifier(criterion='entropy')
model.fit(X, y)

# New Sample: Sunny, Cool, High, Strong
sample = [[2, 0, 0, 0]] # Encoded values

result = model.predict(sample)

print("Prediction:", "Yes" if result[0] == 1 else "No")

```

```

sample = [[2, 0, 0, 0]] # Encoded values

result = model.predict(sample)

print("Prediction:", "Yes" if result[0] == 1 else "No")

... Prediction: No

```

4. Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

```

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

# Load dataset
X, y = load_iris(return_X_y=True)

# Split dataset

```

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)

# Build ANN using Backpropagation
model = MLPClassifier(hidden_layer_sizes=(5,), max_iter=1000, random_state=1)
model.fit(X_train, y_train)

# Test the model
print("Accuracy:", model.score(X_test, y_test))

# Predict a sample
print("Prediction:", model.predict([X_test[0]]))
print("Actual:", y_test[0])

```

output:

```

... Accuracy: 0.7777777777777778
    Prediction: [0]
    Actual: 0
    /usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:100: UserWarning:
    warnings.warn(

```

5. Write a program for Implementation of K-Nearest Neighbours (K-NN) in Python

```

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

# Load dataset
X, y = load_iris(return_X_y=True)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1
)

# Create and train K-NN model
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

# Calculate accuracy
accuracy = model.score(X_test, y_test)

# Predict first test sample
prediction = model.predict([X_test[0]])

# Display results

```

```
print("Accuracy: {:.2f}%".format(accuracy * 100))
print("Predicted Class:", prediction[0])
print("Actual Class:", y_test[0])
```

output:

```
.. Accuracy: 97.78%
   Predicted Class: 0
   Actual Class: 0
```

6. Write a program to implement Naïve Bayes algorithm in python and to display the results using confusion matrix and accuracy.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score

# Load dataset
X, y = load_iris(return_X_y=True)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1
)

# Train Naïve Bayes model
model = GaussianNB()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Results
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nAccuracy: {:.2f}%".format(accuracy_score(y_test, y_pred) * 100))
```

output:

```
.. Confusion Matrix:
   [[14  0  0]
    [ 0 16  2]
    [ 0  1 12]]

   Accuracy: 93.33%
```